



Lecture 03: Numerical Computation

Deep Learning (DL)

EMPOWERING EXPERTISE.

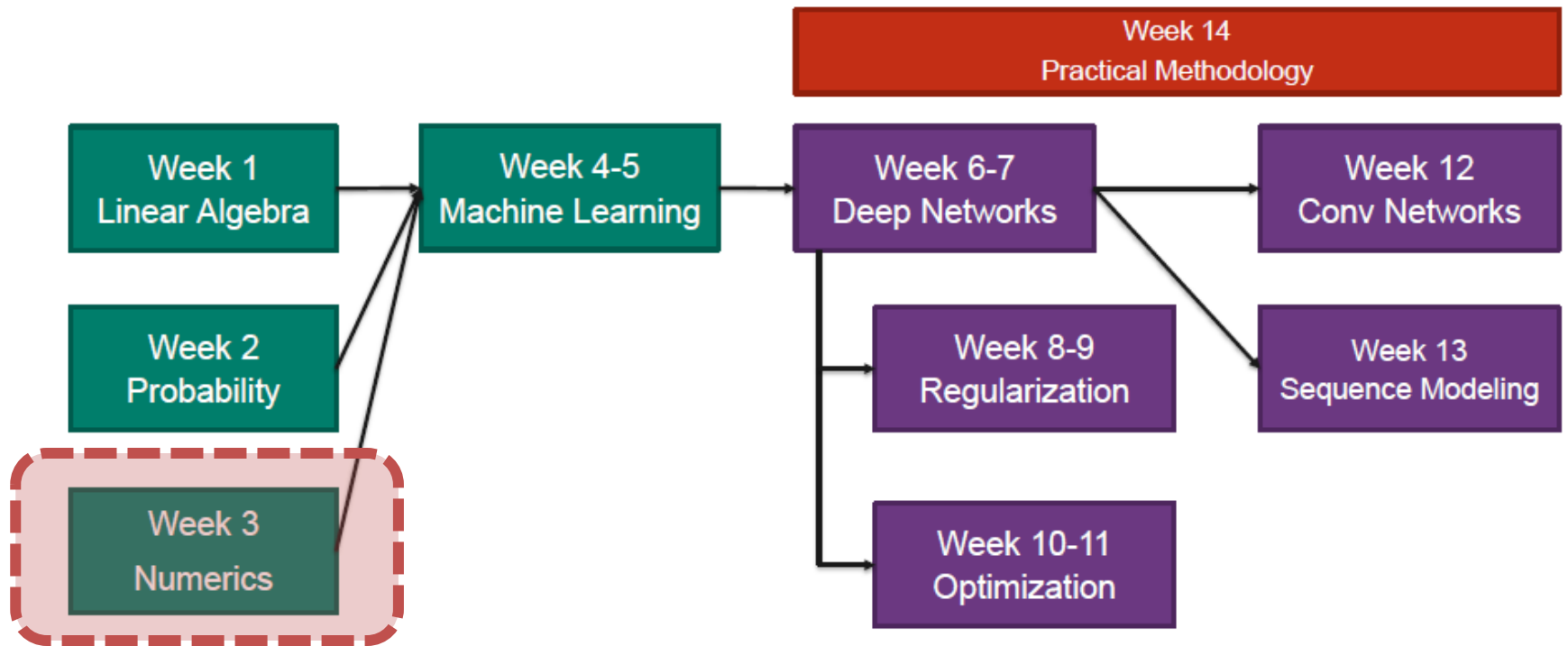


Hannes Badertscher

2 March 2026



Overview of DL-Lecture



Overview of this lecture

- Introduction
- Numerical problems
- Gradient-based Optimization
- Jacobian and Hessian-based Optimization

General Methodology

- Given a set of m **training data** points

$$\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}$$

- The **empirical data distribution** is then:

$$\hat{p}(\mathbf{x}) = \frac{1}{m} \sum_{i=1}^m \delta(\mathbf{x} - \mathbf{x}^{(i)})$$

- The **neural network predicts** a distribution
 - Where θ are the learnable parameters

$$p_{\text{model}}(\mathbf{x}; \boldsymbol{\theta})$$

- Training process:** make $p_{\text{model}}(\mathbf{x})$ as similar as possible to $\hat{p}(\mathbf{x})$

- This can be done by **minimizing the KL-divergence**
 - Or equivalently: **minimize the cross-entropy**

$$D_{\text{KL}}(P \| Q)$$

$$H(P, Q)$$

- ➔ **Learning** is an **optimization** problem

Overview of this lecture

- Introduction
- Numerical problems
- Gradient-based Optimization
- Jacobian and Hessian-based Optimization

Quantization

- Infinitely many real numbers exist
- On a computer: only **finite number of bits** available
- Example:
 - Range R of possible values $[-R/2, R/2]$
 - 3 bits $\rightarrow 2^3 = 8$ possible values
 - Step size $Q = R/8$
- **$\rightarrow$ Rounding errors**
 - Accumulation of rounding errors can be highly problematic

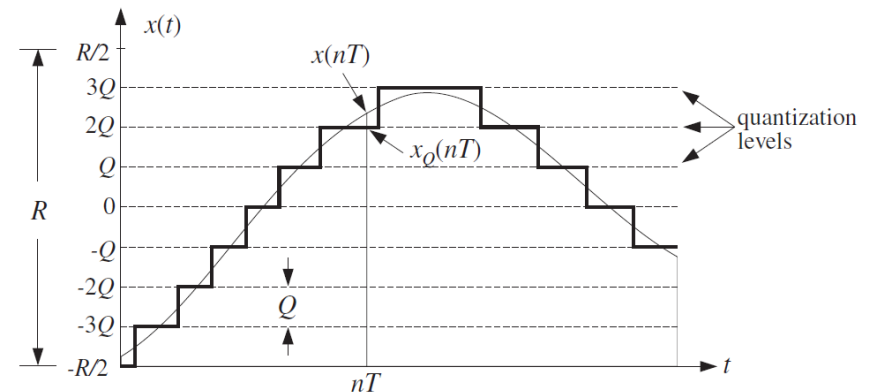


Fig. 2.1.2 Signal quantization.

S.J. Orfanidis, *Introduction to Signal Processing*.
2010. Available: <https://ece.rutgers.edu/orfanidis>

Overflow and Underflow

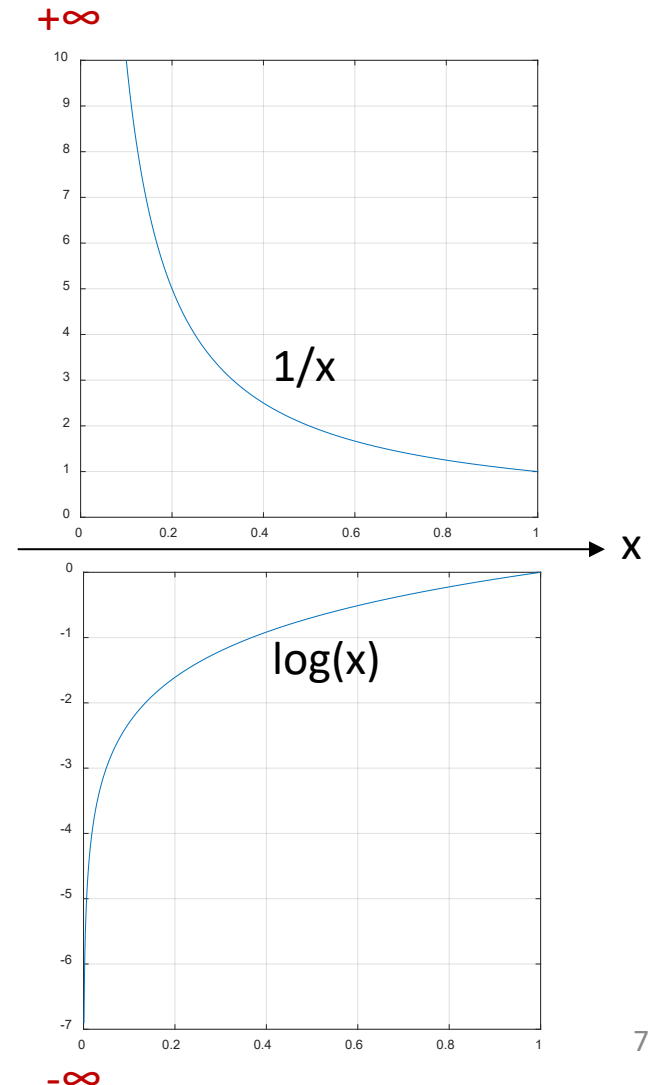
- **Underflow**

- Rounding of small numbers to zero
- ➔ not a direct problem, but further computation may be problematic
- Example: applying $1/x$ or $\log(x)$ to $x=0$ leads to an overflow

- **Overflow**

- Rounding of large numbers to ∞
- ➔ no further computation possible

- ➔ Functions that are prone to overflow or underflow must be **stabilized** against that!



Example: Softmax function

- **Softmax** = extension of sigmoid to **multiple classes**
 - Useful for estimating **parameter vector** of a **multinoulli** distribution
 - ➔ Often used as **final layer** in a DNN for multi-class classification

$$\sigma(x) = \frac{\exp(x)}{1 + \exp(x)}$$

$$\text{softmax}(\mathbf{x})_i = \frac{\exp(x_i)}{\sum_{j=1}^n \exp(x_j)}$$

!

- Numerical problem:
 - Assume that all K inputs x_i have the same constant value c ➔ all classes equally likely
 - Very **negative** c ➔ $\exp(c)$ **underflows** to 0 ➔ 0/0
 - Very **positive** c ➔ $\exp(c)$ **overflows** ∞ ➔ ∞/∞

$$\mathbf{x} = \begin{bmatrix} c \\ c \\ c \\ \vdots \\ c \end{bmatrix} \quad \text{softmax}(\mathbf{x}) = \begin{bmatrix} 1/K \\ 1/K \\ 1/K \\ \vdots \\ 1/K \end{bmatrix}$$

- ➔ Softmax must be stabilized!

Example: Softmax function

- For simplicity we look at a 2-d example

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

$$\text{softmax}(x_1) = \frac{\exp(x_1)}{\exp(x_1) + \exp(x_2)}$$

- Adding any constant** value k to both inputs does not change the output

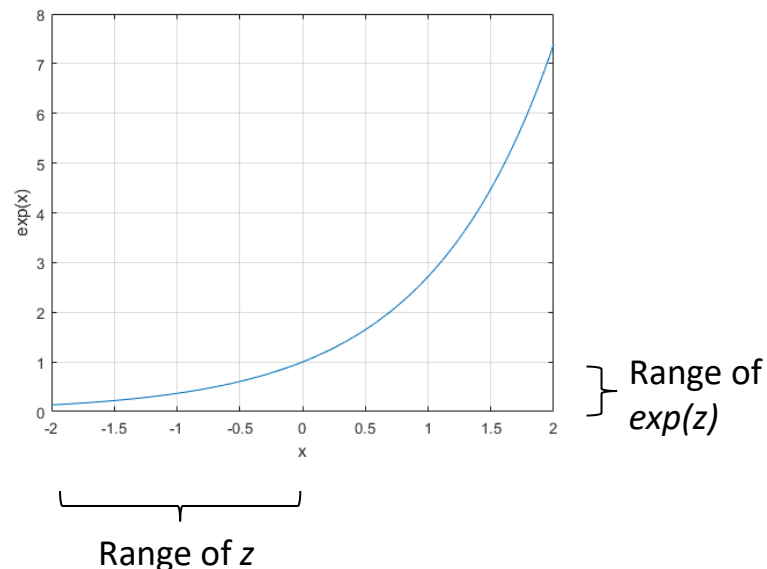
$$\begin{aligned} \text{softmax}(x_1) &= \frac{\exp(x_1 + k)}{\exp(x_1 + k) + \exp(x_2 + k)} \\ &= \frac{\exp(x_1) \exp(k)}{\exp(x_1) \exp(k) + \exp(x_2) \exp(k)} \\ &= \frac{\exp(x_1) \exp(k)}{(\exp(x_1) + \exp(x_2)) \exp(k)} \\ &= \frac{\exp(x_1) \exp(k)}{(\exp(x_1) + \exp(x_2)) \exp(k)} \end{aligned}$$

Example: Softmax function

- The softmax is stabilized by **subtracting the maximum input** from all elements
 - $\rightarrow$ does not change the result of the softmax
 - But, does it help?
- Effects of subtracting the maximum
 - Subtracting the largest entry from all entries means, that the **largest entry is now 0**.
 - All **other entries** will be **negative**
 - At least one input $x_i = 0 \rightarrow \exp(0) = 1 \rightarrow$ denominator > 1 , hence **no 0/0 problem**
 - Largest input = 0 $\rightarrow$ **no overflow problem**

$$z = x - \max_i x_i$$

$$\text{softmax}(x)_i = \frac{\exp(x_i)}{\sum_{j=1}^n \exp(x_j)}$$



Example: Softmax function

- **Additional problem:** often, the logarithm of the softmax is used
 - The stabilized softmax can still underflow to 0 $\rightarrow \log(0) \rightarrow -\infty$
- A **separate stabilization** of the log softmax is required
- **Good news:** today's frameworks do this automatically for you
 - But: if you don't use such tools, beware!

$$\log \text{softmax}(\mathbf{x})$$

Poor Conditioning

Definition of Poor Conditioning:

Small change of the input has large effect on the output

- → small errors (noise, rounding errors) have large effect
- **Conditioning of matrix inverse**
 - Poor conditioning → small errors in x lead to large error in $f(x)$
- **Example:**
 - linear regression, where
 - A = measured inputs
 - x = measured outputs
 - $f(x)$ = parameter vector β

$$f(x) = A^{-1}x$$

Poor Conditioning

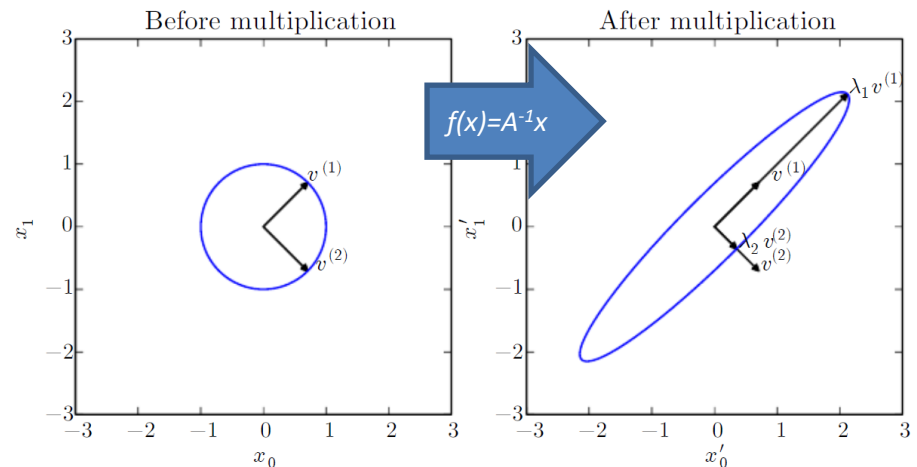
- **Condition number** → measurement of the conditioning

$$f(x) = A^{-1}x$$

- **Definition:** ratio of the **largest eigenvalue** over the **smallest eigenvalue**
 - Well conditioned: ≈ 1
 - Ill conditioned: $\gg 1$ (e.g. $> 10'000$)
 - **Note:** that A and A^{-1} have (under reasonable assumptions) the same conditioning number

$$\max_{i,j} \left| \frac{\lambda_i}{\lambda_j} \right| !$$

- **Visual interpretation**
 - Example: condition number = 6
 - Condition number = 1 → errors stay constant in all directions
 - Condition number $\gg 1$ → errors are amplified in one direction



DL, Fig. 2.3

Overview of this lecture

- Introduction
- Numerical problems
- Gradient-based Optimization
- Jacobian and Hessian-based Optimization

Gradient-Based Optimization

Learning = Minimizing a Loss Function

- In ML: **cross-entropy** between **empirical distribution** and parametric **model distribution**

$$\min_{\theta} H(\hat{p}_{\text{data}}, p_{\text{model}}(\theta)) \quad !$$

- **Notation:**
 - Where $\mathbf{x}$ is a vector of all **learnable parameters**.
 - Often, we also denote the learnable parameters as θ
- The **optimal value** that minimizes the loss function $f(\mathbf{x})$ is denoted as $\mathbf{x}^*$

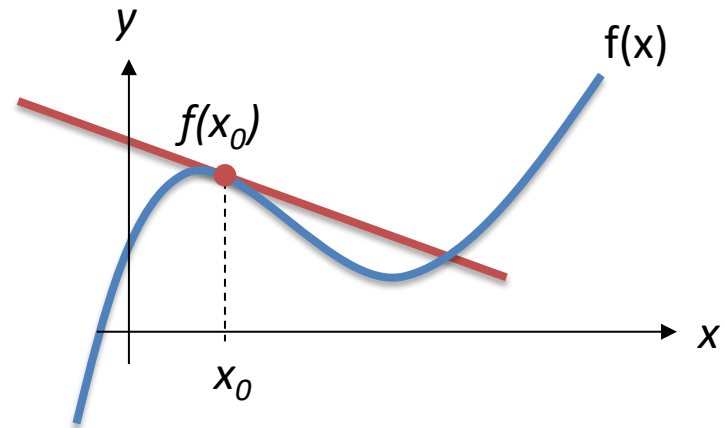
$$f(\mathbf{x})$$

$$\mathbf{x}^* = \arg \min f(\mathbf{x})$$

Minimizing a scalar function

- Simplest case: scalar input x , scalar output y
- **Key idea:**
 - **Derivative** of $f(x)$ = **slope** of the function
 - To minimize, move in the direction of the slope $\rightarrow f(x)$ will decrease!
- The (approximate) new **output**, given a **small change** ϵ at the **input** is given by:
 - If $f'(x)$ negative $\rightarrow f(x + \epsilon)$ will be smaller

$$y = f(x)$$

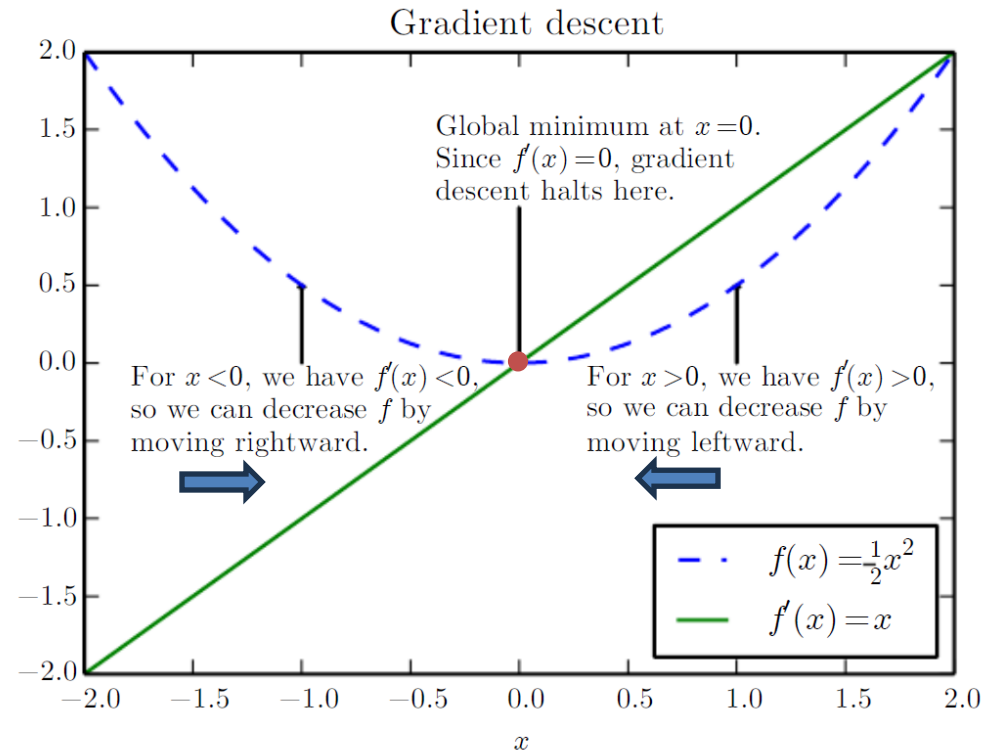


$$f(x + \epsilon) \approx f(x) + \epsilon f'(x) \quad !$$

Gradient-Based Optimization

Algorithm for Gradient Descent

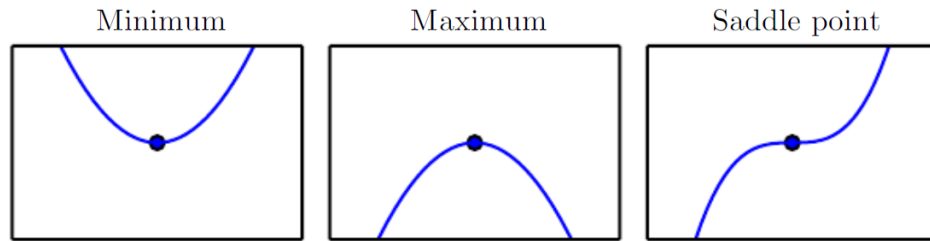
- 1) Initialize to some point x_0
- 2) Calculate derivative $f'(x_0)$
- 3) Take small step ϵ in the negative direction of the derivative
 - $x_1 = x_0 + \epsilon f'(x_0)$
 - $f(x_1) \approx f(x_0) + \epsilon f'(x_0)$
 - $\rightarrow$ if $f'(x_0)$ negative, this will decrease
- 4) Iterate until $f'(x_0) = 0$



DL, Fig. 4.1

Critical points

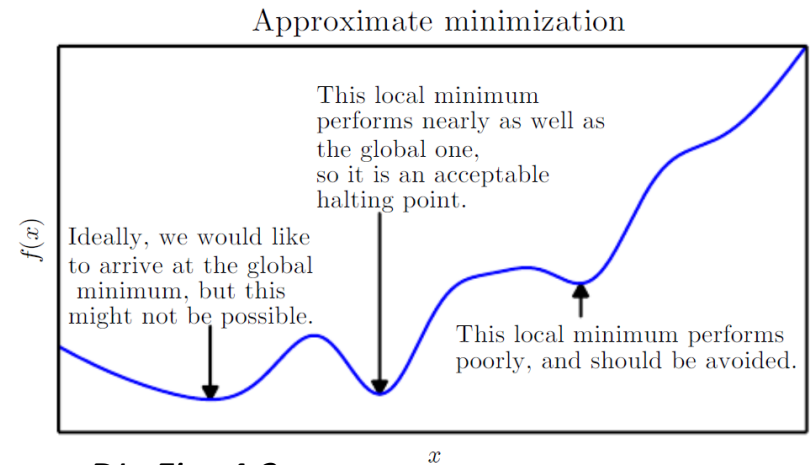
- Critical points (stationary points) are points where $f'(x) = 0$
 - → gradient descent does not know in which direction to go
 - → terminates
- 3 kinds of critical points
 - **Local minimum** → both directions go “up”
 - **Local maximum** → both directions go “down”
 - **Saddle point** → neither, but still $f'(x) = 0$



DL, Fig. 4.2

Local and global minima

- **Gradient descent** can find a **local minimum**
 - There may be many local minima
- **Global minima** are points, that obtain the **absolute lowest value** of $f(x)$
 - There can be one or many local minima (with the same value)
- In **deep learning: many** non-optimal **local minima** and **many saddle points**
 - ➔ usually, we never find the global minimum
 - But: no problem! Goal: be “good enough”



DL, Fig. 4.3

Extension to multiple dimensions

- Vector input $\mathbf{x}$ but scalar output y

$$f : \mathbb{R}^n \rightarrow \mathbb{R} \quad y = f(\mathbf{x})$$

- **Partial derivative** of $f(\mathbf{x})$ with respect to a single input x_i

$$\frac{\partial}{\partial x_i} f(\mathbf{x})$$

- Interpretation: **slope** of $f(\mathbf{x})$ when **changing x_i** but keeping **all other inputs fixed**

- **Gradient** of $f(\mathbf{x})$

$$\nabla_{\mathbf{x}} f(\mathbf{x})$$

- Vector that contains all partial derivatives
- **Size** of gradient: same as size of $\mathbf{x}$ → same as **number of parameters** to optimize
- For example, for a 3-d vector $\mathbf{x}$, the gradient is also 3-d

$$\nabla_{\mathbf{x}} f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f(\mathbf{x})}{\partial x_1} \\ \frac{\partial f(\mathbf{x})}{\partial x_2} \\ \frac{\partial f(\mathbf{x})}{\partial x_3} \end{bmatrix}$$

Extension to multiple dimensions

Scalar input x

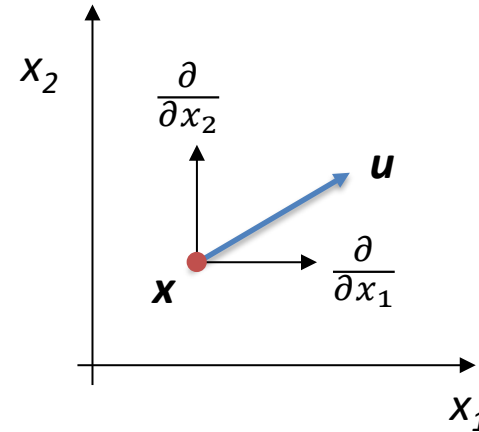
- Scalar derivative: $f'(x)$
- Critical points: $f'(x) = 0$
 - Scalar
- Gradient descent = small steps ϵ in direction where the derivative is negative

Vector input $\mathbf{x}$

- Gradient vector: $\nabla_{\mathbf{x}} f(\mathbf{x})$
- Critical points: $\nabla_{\mathbf{x}} f(\mathbf{x}) = 0$
 - For all elements of $\nabla_{\mathbf{x}} f(\mathbf{x})$
- ➔ in which **direction** to go?

Directional Derivative

- **Partial derivatives** $\rightarrow$ derivatives along directions x_i
- What about any **arbitrary** direction $\mathbf{u}$?
- **Directional derivative**
 - **Definition**
 - **Inner product** between **directional vector** $\mathbf{u}$ and **gradient** $\nabla_x f(\mathbf{x})$
 - Note: this can easily be derived by applying the chain rule to the definition



$$\frac{\partial}{\partial \alpha} f(\mathbf{x} + \alpha \mathbf{u}) \quad \text{at } \alpha = 0$$

$$\frac{\partial}{\partial \alpha} f(\mathbf{x} + \alpha \mathbf{u}) = \mathbf{u}^\top \nabla_x f(\mathbf{x}) \quad !$$

$$\begin{aligned} \frac{\partial f(\mathbf{x} + \alpha \mathbf{u})}{\partial \alpha} &= \frac{\partial (\mathbf{x} + \alpha \mathbf{u})^\top}{\partial \alpha} \frac{\partial f(\mathbf{x} + \alpha \mathbf{u})}{\partial (\mathbf{x} + \alpha \mathbf{u})} \\ &= \mathbf{u}^\top \nabla_x f(\mathbf{x}) \end{aligned}$$

Gradient-Based Optimization

- Back to the **question**: given the gradient, in **which direction** to move to **minimize** $f(\mathbf{x})$?

- $\rightarrow$ minimize directional derivative

- Constraint: $\mathbf{u}^T \mathbf{u} = 1$
 $\rightarrow$ unit length vector

- With $\mathbf{a}^T \mathbf{b} = \|\mathbf{a}\|_2 \|\mathbf{b}\|_2 \cos \theta$

- Constraint: $\|\mathbf{u}\|_2 = 1$

- Gradient is independent of $\mathbf{u} \rightarrow$ constant

$$\min \frac{\partial}{\partial \alpha} f(\mathbf{x} + \alpha \mathbf{u})$$

$$= \min_{\mathbf{u}, \mathbf{u}^T \mathbf{u} = 1} \mathbf{u}^T \nabla_{\mathbf{x}} f(\mathbf{x})$$

$$= \min_{\mathbf{u}, \mathbf{u}^T \mathbf{u} = 1} \|\mathbf{u}\|_2 \|\nabla_{\mathbf{x}} f(\mathbf{x})\|_2 \cos \theta$$

$$= \min_{\mathbf{u}} \cos \theta$$

- The minimum is where $\cos \theta = -1$ which is at $\theta = 180^\circ$
- $\rightarrow$ **opposite direction of gradient**

Gradient-Based Optimization

- Gradient → direction of steepest ascent
- Negative gradient → direction of steepest descent
- **Gradient descent** → small step ϵ in **negative gradient direction**
- **Learning rate ϵ**
 - **Critical parameter** of gradient descent
 - Too small → learning takes forever
 - Too large → chance of “stepping over” a minimum
 - **Smart algorithms** to choose correctly → will be discussed later in the semester

$$x' = x - \epsilon \nabla_x f(x) !$$

Overview of this lecture

- Introduction
- Numerical problems
- Gradient-based Optimization
- Jacobian and Hessian-based Optimization

Jacobian Matrices

- **Extension:** functions with vector-input and **vector-output**

- Note: m must not be equal to n

$$f: \mathbb{R}^m \rightarrow \mathbb{R}^n$$

- There are now $n \times m$ **partial derivatives**

- From n outputs to m inputs

$$\frac{\partial f_i}{\partial y_j}$$

- **Jacobian matrix:** matrix of all partial derivatives

- ➔ extension of the gradient
- ➔ row 1 = gradient of $f_1(\mathbf{x})$

$$\mathbf{J} \in \mathbb{R}^{n \times m}$$

$$J_{i,j} = \frac{\partial}{\partial x_j} f(\mathbf{x})_i \quad !$$

- Typical application: Softmax function

e.g. for $n=m=2$:

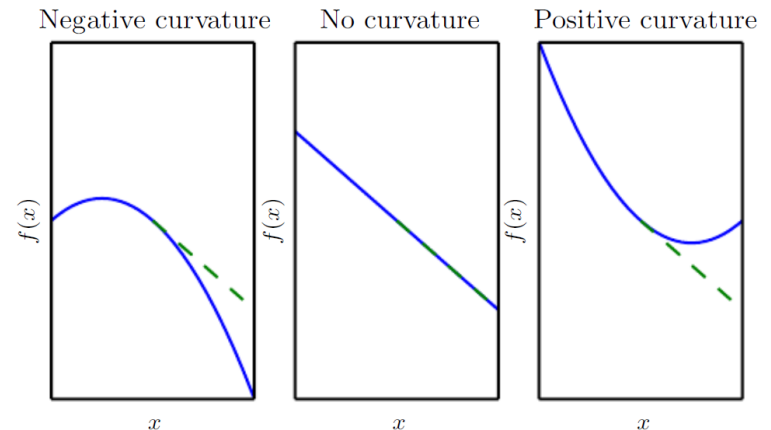
$$\mathbf{J} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} \end{bmatrix}$$

Curvature / Second Derivatives

- Back to **scalar functions** $f(\mathbf{x})$ of vector arguments $\mathbf{x}$
- **Second derivative:** derivative of the derivative
 - Notation: $f''(\mathbf{x})$
 - Interpretation: slope of the slope
- **Curvature**
 - Slope of slope = 0 = constant $\rightarrow$ straight line, no curvature
 - Slope of slope $> 0 \rightarrow$ slope increases with $x \rightarrow$ positive curvature
 - Slope of slope $< 0 \rightarrow$ slope decreases with $x \rightarrow$ negative curvature

$$f : \mathbb{R}^n \rightarrow \mathbb{R}$$

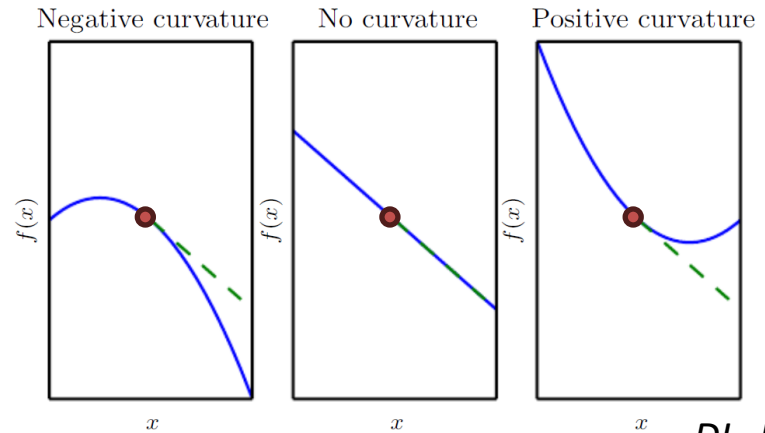
$$\frac{\partial^2}{\partial x_i \partial x_j} f$$



DL, Fig. 4.4

Curvature / Second Derivatives

- **Application of curvature**
 - Gradient descent assumes that $f(x_0 + \epsilon) \approx f(x_0) + \epsilon f'(x_0)$
 - If curvature = 0 $\rightarrow$ approximation holds exactly
 - If curvature > 0 $\rightarrow$ slope increase $\rightarrow$ overly optimistic / too large steps
 - If curvature < 0 $\rightarrow$ slope decreases $\rightarrow$ overly pessimistic / too small steps



DL, Fig. 4.4

Curvature	Slope	Gradient descent step
Negative	Gets smaller	Too small
Zero	Stays constant	Just right
Positive	Gets larger	Too large

Hessian Matrices

- **Second derivative** → derivative of derivative
 - First derivative: Gradient vector
 - Second derivative: **Jacobian matrix** of the **gradient**

$$H(f)(x)_{i,j} = \frac{\partial^2}{\partial x_i \partial x_j} f(x) \quad !$$

- The matrix of **second derivatives** is called the **Hessian matrix**

e.g. for $n=2$:

$$H = \begin{bmatrix} \frac{\partial^2}{\partial x_1^2} f & \frac{\partial^2}{\partial x_1 \partial x_2} f \\ \frac{\partial^2}{\partial x_2 \partial x_1} f & \frac{\partial^2}{\partial x_2^2} f \end{bmatrix}$$

- Note: the **Hessian matrix** is a **symmetric** matrix
 - → **Eigendecomposition** with real eigenvalues possible

$$\frac{\partial^2}{\partial x_i \partial x_j} f(x) = \frac{\partial^2}{\partial x_j \partial x_i} f(x)$$

$$H_{i,j} = H_{j,i}$$

Directional second derivative

- **First directional derivative** in any direction $\mathbf{d}$
- **Second directional derivative** in any direction $\mathbf{d}$
- Because H is symmetric, it has an **eigendecomposition**
 - V : matrix containing all **eigenvectors**
 - Λ : diagonal matrix of **eigenvalues** λ

$$\mathbf{d}^T \nabla_x f(\mathbf{x})$$

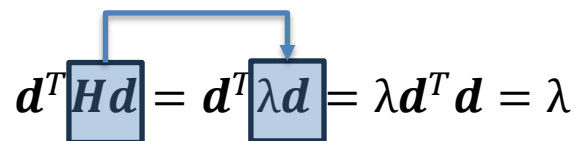
$$\mathbf{d}^T H \mathbf{d}.$$
 !

$$H = V \Lambda V^T$$

Directional second derivative

- If $\mathbf{d}$ is an **eigenvector**, the **second directional derivative** is

- $\rightarrow$ **Eigenvalues of H** are the **curvature** in direction of the eigenvectors


$$\mathbf{d}^T \boxed{H} \mathbf{d} = \mathbf{d}^T \boxed{\lambda} \mathbf{d} = \lambda \mathbf{d}^T \mathbf{d} = \lambda$$

- For all **other directions**, curvature is a **weighted sum of eigenvalues**

- Where θ_i is the angle between $\mathbf{d}$ and the i -th eigenvector $\mathbf{v}_i$
- $(\cos \theta_i)^2$ is **maximal** if $\theta_i = 0 \rightarrow$ in the direction of the **corresponding Eigenvector**
- $\rightarrow$ **Maximum curvature** is λ_{max} in the direction of $\mathbf{v}_{max}$
- $\rightarrow$ **Minimum curvature** is λ_{min} in the direction of $\mathbf{v}_{min}$

$$\mathbf{d}^T H \mathbf{d} = \mathbf{d}^T V \Lambda V^T \mathbf{d} = \sum_i (\mathbf{d}^T \mathbf{v}_i)^2 \lambda_i$$

$$= \sum_i (\cos \theta_i)^2 \lambda_i \quad !$$

2nd order Taylor approximation

- **Taylor approximation** → **approximation** of any function $f(x)$ by a **polynomial**

- 2nd order → polynomial of order 2

- Example in 1-d:
$$f(x) \approx f(x^{(0)}) + \frac{f'(x^{(0)})}{1!}(x - x^{(0)}) + \frac{f''(x^{(0)})}{2!}(x - x^{(0)})^2 + \dots$$

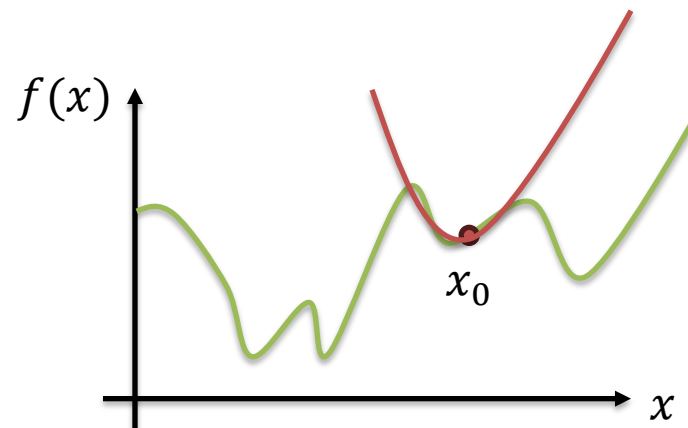
- Example in n-d:

- where:

- $\mathbf{g}$: gradient
- $\mathbf{H}$: Hessian

$$f(\mathbf{x}) \approx f(\mathbf{x}^{(0)}) + (\mathbf{x} - \mathbf{x}^{(0)})^\top \mathbf{g} + \frac{1}{2}(\mathbf{x} - \mathbf{x}^{(0)})^\top \mathbf{H}(\mathbf{x} - \mathbf{x}^{(0)}) \quad !$$

- As long as $\mathbf{x}$ stays **close to** $\mathbf{x}^{(0)}$, this approximation is usually **good**



2nd order Taylor approximation

- **Analyzing gradient descent** with a 2nd order Taylor approximation
 - Gradient descent makes a **small step** ϵ in the negative gradient direction
 - Taylor approximation at $\mathbf{x}^{(0)}$ $\rightarrow$ estimate $f(\mathbf{x}^{(0)} - \epsilon \mathbf{g})$

$$\mathbf{x}^{(0)} - \epsilon \mathbf{g}$$

- **Result: 3 parts**

- $f(\mathbf{x}^{(0)}) \rightarrow$ old point
- $-\epsilon \mathbf{g}^T \mathbf{g} \rightarrow$ small negative number
- $\frac{1}{2} \epsilon^2 \mathbf{g}^T \mathbf{H} \mathbf{g} \rightarrow$ see next slide

$$f(\mathbf{x}^{(0)} - \epsilon \mathbf{g}) \approx \boxed{f(\mathbf{x}^{(0)})} - \boxed{\epsilon \mathbf{g}^T \mathbf{g}} + \boxed{\frac{1}{2} \epsilon^2 \mathbf{g}^T \mathbf{H} \mathbf{g}} !$$

- **First two parts are guaranteed to decrease $f(x)$**

2nd order Taylor approximation

- **Effect of curvature term**

- If $\mathbf{g}^T \mathbf{H} \mathbf{g}$ is **zero or negative**:

- A large ε can be used.
 - In theory, the larger ε , the better.
 - But: as we move far away from $\mathbf{x}^{(0)}$, our approximation becomes worse, so this is not entirely true.

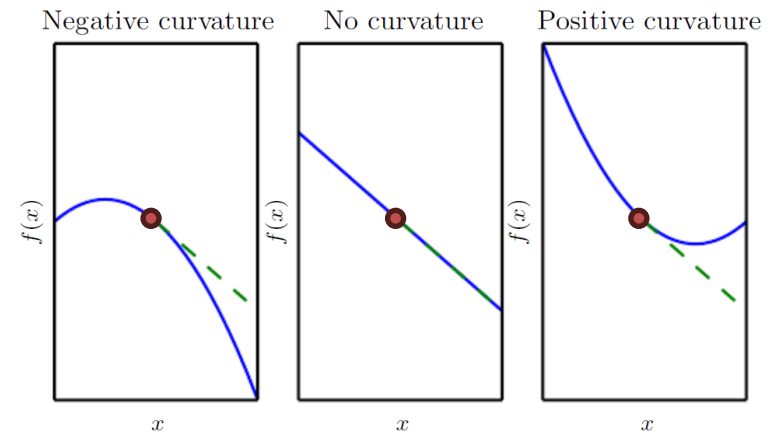
$$f(\mathbf{x}^{(0)} - \varepsilon \mathbf{g}) \approx \boxed{f(\mathbf{x}^{(0)})} - \boxed{\varepsilon \mathbf{g}^T \mathbf{g}} + \boxed{\frac{1}{2} \varepsilon^2 \mathbf{g}^T \mathbf{H} \mathbf{g}}$$

- If $\mathbf{g}^T \mathbf{H} \mathbf{g}$ is **positive**:

- Counters the decrease by the gradient $\rightarrow$ smaller decrease
 - $\rightarrow$ can even increase $f(x)$

- Strong curvature can make gradient descent fail

- Or at least: require **very small steps** ε



DL, Fig. 4.4

Testing for minima or maxima

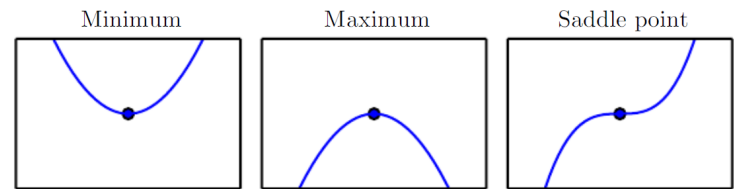
- **Second derivative test in 1D**

- The second derivative can be used to **determine whether a critical point $f'(x)=0$ is a local maximum, a local minimum or a saddle point.**

$f''(x) > 0$

$f''(x) < 0$

$f''(x) = 0$



DL, Fig. 4.2

Type of critical point	$f''(x)$	if $x \uparrow$	if $x \downarrow$
Minimum	> 0	$f'(x) \uparrow$	$f'(x) \uparrow$
Maximum	< 0	$f'(x) \downarrow$	$f'(x) \downarrow$
Saddle Point	$= 0$	-	-

Testing for minima or maxima

- **Second derivative test in higher dimensions**

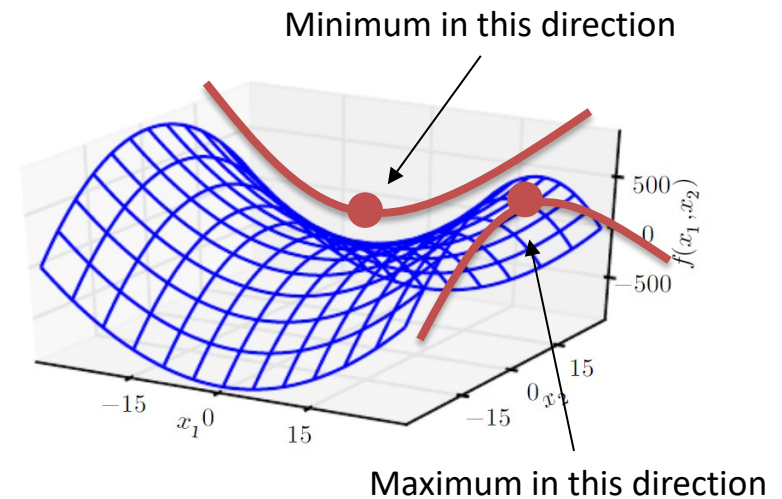
- → more complicated
- **Critical points:** gradient is the zero vector
- **Second derivative:** Hessian matrix
- → consider Eigenvalues of Hessian

$$\nabla_x f(x) = 0$$

Type of critical point	Eigenvalues of H	2nd order directional derivatives
Minimum	All positive	Positive in all possible directions
Maximum	All negative	Negative in all possible directions
Saddle Point	Some positive, some negative	Positive in some, negative in other directions

Saddle points

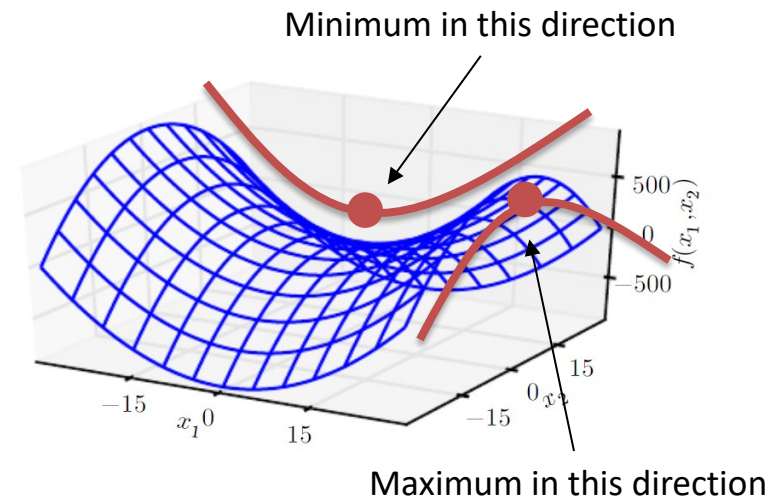
- Second derivative test in higher dimensions
 - If at least one eigenvalue is negative and at least one eigenvalue is positive, it is clear that $\mathbf{x}$ is in one direction a local minimum while in the other direction it is a local maximum → Hence this $\mathbf{x}$ is a saddle point
 - The test might also be inconclusive when all eigenvalues have the same sign but at least one eigenvalue is zero



DL, Fig. 4.5

Saddle points

- If some **eigenvalues** of the **Hessian** are **positive** and some are **negative**, the point is a saddle point
- Example
 - Point is a **minimum in direction x_1** and a **maximum in direction x_2**
 - $\rightarrow \lambda_1$ positive, λ_2 negative
- Problem for **high-dimensional optimization problems**:
 - E.g. 1'000'000 parameters $\rightarrow$ 1'000'000-dimensional space
 - Hessian has 1'000'000 eigenvalues
 - If 999'999 are positive and 1 is negative $\rightarrow$ saddle point and not a minimum

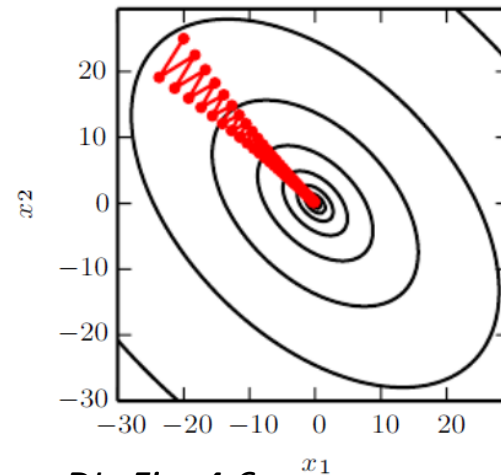


DL, Fig. 4.5

Condition Number

- The **condition number of the Hessian** is a good way of measuring **how much the second derivatives vary**
 - A **small** condition number means...
 - **Curvature** similar in all directions.
 - → large step size ϵ
 - A **high** condition number means...
 - **Large curvature in one direction, small curvature in another direction.**
 - → **small step size** to prevent overshooting in one direction.
 - But: slow progress in other directions
 - This is called “**poor conditioning**”

$$\max_{i,j} \left| \frac{\lambda_i}{\lambda_j} \right|$$



DL, Fig. 4.6

Overview of this lecture

- Introduction
- Numerical problems
- Gradient-based Optimization
- Jacobian and Hessian-based Optimization

Summary: Numerical Computing

- **Numerical problems**
 - Limited precision (quantization)
 - Underflow = small values $\rightarrow 0$
 - Overflow = large values $\rightarrow \infty$

- **Gradient-based optimization**

- Goal: minimize a function $f(\mathbf{x})$

$$\mathbf{x}^* = \arg \min f(\mathbf{x})$$

- Gradient: vector of all partial derivatives of $f(\mathbf{x})$

$$\nabla_{\mathbf{x}} f(\mathbf{x}) \quad \frac{\partial}{\partial x_i} f(\mathbf{x})$$

- Directional derivative in direction $\mathbf{u}$

$$\frac{\partial}{\partial \alpha} f(\mathbf{x} + \alpha \mathbf{u}) = \mathbf{u}^\top \nabla_{\mathbf{x}} f(\mathbf{x}) \quad \text{at } \alpha = 0$$

- Gradient descent / steepest descent
follow the negative gradient in small steps ϵ

$$\mathbf{x}' = \mathbf{x} - \epsilon \nabla_{\mathbf{x}} f(\mathbf{x})$$

- **Jacobian** = generalization of gradient to vector-valued functions $\rightarrow$ matrix

$$J_{i,j} = \frac{\partial}{\partial x_j} f(\mathbf{x})_i$$

Summary: Numerical Computing

- **Hessian** = matrix of all second-order derivatives $H(f)(x)_{i,j} = \frac{\partial^2}{\partial x_i \partial x_j} f(x)$
 - **Eigendecomposition of the Hessian** $d^\top H d$.
 - Hessian contains a large number of values
 - To analyze it more easily, the Eigenvalues are analyzed
 - **Second order Taylor approximation**

$$f(x) \approx f(x^{(0)}) + (x - x^{(0)})^\top g + \frac{1}{2} (x - x^{(0)})^\top H (x - x^{(0)})$$
 - Locally approximates a function $f(x)$ around a given point $x^{(0)}$ using the gradient g and Hessian H
 - The approximated value after one step of gradient descent is then

$$f(x^{(0)} - \epsilon g) \approx f(x^{(0)}) - \epsilon g^\top g + \frac{1}{2} \epsilon^2 g^\top H g$$
 - **Second derivative test**
 - Uses the second derivative $f''(x)$ or the Eigenvalues of the Hessian to judge whether a critical point is a minimum, maximum, or saddle point

Type of critical point	$f''(x)$	Eigenvalues of H
Minimum	> 0	All positive
Maximum	< 0	All negative
Saddle Point	$= 0$	both

Overview of DL-Lecture

